

포팅메뉴얼

0. 사전 준비

0-1. 프로젝트 클론

```
git clone <프로젝트 Git 주소>
```

0-2. Android Studio 설치

1. 환경 변수 정리

1-1. 로컬 개발용 환경 변수

1) Backend

```
# PostgreSQL
DB_URL=jdbc:postgresql://<PostgreSQL DB 서버 주소>:<포트번호>/<PostgreSQL DB 이름>
DB_USERNAME=<PostgreSQL 유저 이름>
DB_PASSWORD=<PostgreSQL DB 비밀번호>

# Redis
REDIS_HOST=<Redis 서버 주소>
REDIS_PORT=<Redis 서버 포트>
REDIS_PASSWORD=<Redis 비밀번호>

# GMS 연동
GMS_API_KEY=<GMS API 키>
GMS_BASE_URL=<GMS 기본 URL>
GMS_PATH_OPENAI=<GMS OPENAI PATH>
```

2) Frontend

```
VITE_API_ORIGIN=<API 요청 기본 URL(백엔드 서버 주소, 예: http://localhost:8080)>
VITE_KAKAO_MAP_APP_KEY=<카카오맵 API 키>
```

1-2. GitLab CI Variables

- GitLab → 프로젝트 → Settings → CI/CD → Variables에 환경 변수 등록한다.

1) Backend

```
DEV_DB_NAME=<PostgreSQL DB 이름>
DEV_DB_USER=<PostgreSQL 유저 이름>
DEV_DB_PASSWORD=<PostgreSQL DB 비밀번호>

DEV_REDIS_HOST=<Redis 컨테이너 이름 또는 호스트>
DEV_REDIS_PORT=<Redis 포트>
DEV_REDIS_PASSWORD=<Redis 비밀번호>

GMS_API_KEY=<GMS API 키>
GMS_BASE_URL=<GMS 기본 URL>
GMS_PATH_OPENAI=<GMS OPENAI PATH>
```

2) Frontend

```
VITE_API_ORIGIN=<API 요청 기본 URL>
VITE_KAKAO_MAP_APP_KEY=<카카오맵 API 키>
```

2. 로컬 구동

2-1. Backend 로컬 실행

1) PostgreSQL/Redis 준비

- 직접 설치했을 경우 `DB_URL`, `REDIS_HOST`, `REDIS_PORT`에 맞게 값을 맞추면 된다.
- 도커로 띄우고 싶을 경우 개발자 로컬에서 아래와 같이 입력한다.

```
export DEV_DB_NAME=<PostgreSQL DB 이름>
export DEV_DB_USER=<PostgreSQL 유저 이름>
export DEV_DB_PASSWORD=<PostgreSQL DB 비밀번호>
export DEV_REDIS_HOST=<Redis 컨테이너 이름 또는 호스트>
export DEV_REDIS_PORT=<Redis 포트>
export DEV_REDIS_PASSWORD=<Redis 비밀번호>

docker compose -p runningcity-dev -f deploy/docker-compose.dev.yaml
l --profile infra up -d postgres-dev redis-dev
```

2) Backend 환경 변수 세팅

- 셸에서 직접 입력할 경우

```
export DB_URL=jdbc:postgresql://localhost:5432/<DB 이름>
export DB_USERNAME=<DB 유저>
export DB_PASSWORD=<DB 비밀번호>
export REDIS_HOST=localhost
export REDIS_PORT=6379
export REDIS_PASSWORD=<Redis 비밀번호>

export GMS_API_KEY=<GMS API 키>
export GMS_BASE_URL=<GMS 기본 URL>
export GMS_PATH_OPENAI=<GMS OPENAI PATH>
```

- 또는 IDE 변수 입력, `application.yml` 직접 입력한다.

3) Spring Boot 실행

```
cd Backend
./gradlew bootRun
```

2-2. Frontend 로컬 실행

1) 환경 변수 (`.env.local`)

- `Frontend/.env.local`

```
VITE_API_ORIGIN=<로컬 백엔드 주소>  
VITE_KAKAO_MAP_APP_KEY=<카카오맵 API 키>
```

2) 안드로이드 웹뷰 호환 설정

- Frontend/vite.config.ts

```
allowedHosts: [  
  "nonconjunctive-cami-outdoor.ngrok-free.dev",  
  "localhost",  
  "<IP 주소>", // 각자의 IP 주소로 수정  
  "10.0.2.2",  
],  
hmr: {  
  protocol: "ws",  
  host: "<IP 주소>", // 각자의 IP 주소로 수정  
  clientPort: 5173,  
},
```

- 000.000.000.000 형태의 IP 주소 입력

2) 의존성 설치 & 구동

```
cd Frontend  
npm install  
npm run dev
```

3) 백엔드 연동 확인

- 브라우저에서 <http://localhost:5173> (Vite 설정에 따라 포트 변동)
- 네트워크 탭에서 VITE_API_ORIGIN 으로 요청이 제대로 나가는지 확인 한다.

3-1. Android 앱 실행

1) Android SDK / Emulator 또는 실제 디바이스 연결

- 프로젝트 개발 기준 환경
 - 모바일 Pixel 8(Android 16.0)

- Wear OS Large Round(Android 13.0)

2) 프로젝트 열기

- Android Studio → Open → 클론한 프로젝트의 안드로이드 디렉토리 선택 (`/android`)

3) Android Studio Gradle SDK 버전 변경

- Settings → Build, Execution, Deployment → Build Tools → Gradle → Gradle JDK 경로 수정
- JDK 17로 업데이트

4) 모바일 웹뷰 업데이트 (웹뷰 렌더링 에러시 선택)

- 구동하는 디바이스의 웹뷰 버전이 낮을 경우
- 모바일 디바이스(또는 Emulator) 구글 플레이 스토어 → Android System Webview Dev 설치 및 실행
- Change WebView Provider → Android System WebView Dev 선택

5) 로컬 IP 연결

- `/android/app/src/build.gradle.kts`

```
debug {
    // 디버그 빌드용 URL 추가 (로컬호스트:5173 사용)
    buildConfigField("String", "WEBVIEW_URL", "\"http://<IP 주소>:5173\"") // 또는 각자의 IP 주소를 사용하기
}
```

- `000.000.000.000` 형태의 IP 주소 입력
- 실제 서버 연결시 배포 주소 입력

6) Crashlytics `google-services.json` 추가

- 발급받은 `google-services.json` 파일을 `app/src` 경로에 추가

7) 모바일 앱 실행

- 상단 Run Configuration 에서 `app` 모듈 선택
- 실행 대상 디바이스(에뮬레이터 / 실제 기기) 선택

- Run 버튼 클릭

8) 웨어 앱 실행

- Run Configuration 에서 `wearos` 모듈 선택
- Wear OS 에뮬레이터 또는 실제 워치 선택
- Run 버튼 클릭

9) 모바일-웨어 연동

- 워치 연동
- 애플레이터 연동시, Device manager → 연동 기기 선택 → Pair Wearable → 상대 기기 선택

3. 서버 배포

3-1. 서버 사전 준비(최초 1회)

1) Docker/Compose 설치

```
sudo apt update && sudo apt install -y ca-certificates curl gnupg jq
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker ubuntu

sudo mkdir -p /usr/local/lib/docker/cli-plugins
curl -SL https://github.com/docker/compose/releases/download/v2.29.7/docker-compose-linux-x86_64 \
| sudo tee /usr/local/lib/docker/cli-plugins/docker-compose > /dev/null
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
```

2) GitLab Runner 설치 및 설정

```
curl -L https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh | sudo bash
sudo apt install -y gitlab-runner
sudo systemctl status gitlab-runner
```

```

sudo gitlab-runner register
# <GitLab URL> 입력
# <토큰 입력>
# <러너 이름> 입력
# <executor> 입력 (shell)

# 러너에 도커 권한 추가
sudo usermod -aG docker gitlab-runner
sudo systemctl restart gitlab-runner

```

3) 디렉토리 생성 & 권한

```

# 프론트 Blue-Green 릴리스 저장소
$ sudo mkdir -p /srv/frontend/releases /srv/frontend/shared /srv/frontend/
current

# 백엔드용 디렉토리(필요 시)
$ sudo mkdir -p /srv/backend

# 상점 에셋 저장소
$ sudo mkdir -p /srv/store-assets

# GitLab Runner 권한
$ sudo chown -R gitlab-runner:gitlab-runner /srv/frontend /srv/store-asset
s

# 권한 표준화(디렉토리 755, 파일 644)
$ sudo find /srv/store-assets -type d -exec chmod 755 {} \;
$ sudo find /srv/store-assets -type f -exec chmod 644 {} \;

```

4) Blue-Green 스위치/정리 스크립트

- `/usr/local/bin/fe_switch.sh`

```

sudo tee /usr/local/bin/fe_switch.sh >/dev/null <<'SH'
#!/usr/bin/env bash
set -eou pipefail

```

```

REL="${1:?usage: fe_switch <release> [project_dir]}"
PROJ_DIR="${2:-${CI_PROJECT_DIR:-$PWD}}"
COMPOSE_FILE="$PROJ_DIR/deploy/docker-compose.dev.yml"

export COMPOSE_PROJECT_NAME="${COMPOSE_PROJECT_NAME:-runningcity-dev}"

test -f "$COMPOSE_FILE" || { echo "Compose not found: $COMPOSE_FILE" >&2; exit 1; }
test -d "/srv/frontend/releases/$REL" || { echo "No release /srv/frontend/releases/$REL" >&2; exit 1; }

# current가 '디렉토리'면 제거(심볼릭 링크로 치환 목적)
if [ -d /srv/frontend/current ] && [ ! -L /srv/frontend/current ]; then
    rm -rf /srv/frontend/current
fi

# 상대 링크로 전환 (컨테이너 내부에서도 경로 유효)
(
    cd /srv/frontend
    ln -sfT "releases/$REL" "current"
)

# nginx 컨테이너 유지/재적용 + reload
docker compose -f "$COMPOSE_FILE" up -d --no-deps --remove-orphans nginx
docker compose -f "$COMPOSE_FILE" exec -T nginx nginx -s reload || true

echo "Switched to $REL (compose: $COMPOSE_FILE, project: $COMPOSE_PROJECT_NAME)"
SH
sudo chmod +x /usr/local/bin/fe_switch.sh

```

- `/usr/local/bin/fe_cleanup.sh`

```

sudo tee /usr/local/bin/fe_cleanup.sh >/dev/null <<'SH'
#!/usr/bin/env bash

```



```
set -e
cd /srv/frontend/releases
ls -lt | tail -n +6 | xargs -r rm -rf
SH
sudo chmod +x /usr/local/bin/fe_cleanup.sh
```

5) 프로젝트 clone

```
cd /srv
git clone <repo> runningcity
cd /srv/runningcity
```

6) gradlew 권한

```
sed -i 's/\r$//' Backend/gradlew || true
chmod +x Backend/gradlew || true
```

7) 터미널 세션에 임시 환경변수 등록

```
export DEV_DB_NAME=<PostgreSQL DB 이름>
export DEV_DB_USER=<PostgreSQL 유저 이름>
export DEV_DB_PASSWORD=<PostgreSQL DB 비밀번호>
export DEV_REDIS_HOST=<Redis 서버 이름>
export DEV_REDIS_PORT=<Redis 서버 포트>
export DEV_REDIS_PASSWORD=<Redis 서버 비밀번호>
export GMS_API_KEY=<GMS API 키>
export GMS_BASE_URL=<GMS 기본 URL>
export GMS_PATH_OPENAI=<GMS OPENAI PATH>
export VITE_API_ORIGIN=<API 요청 기본 URL(백엔드 서버 주소)>
export VITE_KAKAO_MAP_APP_KEY=<카카오맵 API 키>
```

- 또는 서버에 별도 파일 생성하여 사용한다.

3-2. 인프라 최초 구동

```
cd /srv/runningcity
```

```
docker compose -p runningcity-dev -f deploy/docker-compose.dev.yml \
--profile infra up -d postgres-dev redis-dev
```

4. Docker Compose / Nginx 구조

4-1. `docker-compose.dev.yml` 서비스

```
services:
  redis-dev:
    image: redis:7-alpine
    command: ["redis-server", "--requirepass", "${DEV_REDIS_PASSWORD}"]
    environment:
      - REDIS_PASSWORD=${DEV_REDIS_PASSWORD}
    ports:
      - 6379:6379
    profiles: ["infra"]

  postgres-dev:
    image: postgis/postgis:16-3.5
    environment:
      POSTGRES_DB: ${DEV_DB_NAME}
      POSTGRES_USER: ${DEV_DB_USER}
      POSTGRES_PASSWORD: ${DEV_DB_PASSWORD}
    volumes:
      - pgdata_dev:/var/lib/postgresql/data
      - ./db/init:/docker-entrypoint-initdb.d:ro
    ports:
      - 5432:5432
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 20
    profiles: ["infra"]
```

```

backend-dev:
  image: local/backend:develop
  environment:
    SPRING_PROFILES_ACTIVE: dev
    SPRING_DATASOURCE_URL: ${SPRING_DATASOURCE_URL:-jdbc:postg
resql://postgres-dev:5432/${DEV_DB_NAME}}
    SPRING_DATASOURCE_USERNAME: ${DEV_DB_USER}
    SPRING_DATASOURCE_PASSWORD: ${DEV_DB_PASSWORD}
    REDIS_HOST: ${DEV_REDIS_HOST}
    REDIS_PORT: ${DEV_REDIS_PORT}
    REDIS_PASSWORD: ${DEV_REDIS_PASSWORD}
    GMS_API_KEY: ${GMS_API_KEY}
    GMS_BASE_URL: ${GMS_BASE_URL}
    GMS_PATH_OPENAI: ${GMS_PATH_OPENAI}
  restart: unless-stopped

nginx:
  image: nginx:alpine
  volumes:
    - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    - /srv/frontend:/var/www:ro
    - /srv/store-assets:/var/www-store-assets:ro
  ports:
    - 80:80
  depends_on:
    backend-dev:
      condition: service_started
  restart: unless-stopped

volumes:
  pgdata_dev:

```

- `redis-dev`, `postgres-dev` 는 `infra` 프로필에 있어 최초 1회 수동 배포 후 CI에서는 자동 재 배포하지 않는다.

4-2. Nginx 라우팅(`default.conf`)

```

server {
    listen 80;
    server_name _;

    root /var/www/current;
    index index.html;

    # 헬스 체크
    location = /health {
        add_header Content-Type text/plain;
        return 200 'OK';
    }

    # Backend 프록시
    location /api/ {
        proxy_pass http://backend-dev:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Front 빌드 버전 정보
    location = /version.json {
        default_type application/json;
        add_header Cache-Control "no-store";
        try_files /version.json =404;
    }

    # 정적 파일 캐싱
    location ~* \.(js|css|png|jpg|jpeg|gif|svg|ico|woff2?)$ {
        expires 7d;
        add_header Cache-Control "public, max-age=604800, immutable";
        try_files $uri =404;
    }

    # store-assets (아이템 이미지 등)
    location ^~ /store-assets/ {

```

```

alias /var/www-store-assets/;
add_header Cache-Control "public, max-age=31536000, immutable";
limit_except GET { deny all; }
types { image/png png; }
try_files $uri =404;
}

# SPA 라우팅
location / {
    add_header Cache-Control "no-cache";
    try_files $uri /index.html;
}
}

```

- `root /var/www/current` :
Blue-Green 배포 스크립트(`fe_switch.sh`)가 `/srv/frontend/current` → `/var/www/current` 로 연결
- `/api/` → `backend-dev:8080`
Spring `context-path: /api/v1` 이므로 최종 API는 `/api/v1/...`
- `/store-assets/` → `/srv/store-assets`
→ 프론트 `ENV.ASSETS_ORIGIN = "<배포 주소>/store-assets"` 와 매칭.

5. GitLab CI/CD 배포 흐름

5-1. 공통 룰

```

.default_chown:
  before_script:
    - docker run --rm -v "$PWD:/w" alpine:3.20 sh -lc "chown -R $(id -u):$(id -g) /w || true"
  after_script:
    - docker run --rm -v "$PWD:/w" alpine:3.20 sh -lc "chown -R $(id -u):$(id -g) /w || true"

```

- 체크아웃된 소스 및 빌드 산출물이 root 소유로 남지 않도록 정리한다.

5-2. Backend 배포 (`deploy_dev_backend`)

- 트리거 조건: `develop` 브랜치에서 `Backend/**/*`, `deploy/docker-compose.dev.yml`, `.gitlab-ci.yml` 변경 시
- 동작 요약:
 1. Gradle 컨테이너에서 `./gradlew jibBuildTar` 실행 → `Backend/build/jib-image.tar` 생성
 2. `docker load` 로 이미지 로드 후 `local/backend:develop` 태그
 3. `docker compose -f deploy/docker-compose.dev.yml up -d --no-deps backend-dev`
 4. `http://backend-dev:8080/api/v1/actuator/health` 헬스체크

5-3. Frontend 배포 (Blue-Green, `deploy_dev_front_static`)

- 트리거 조건: `develop` 브랜치에서 `Frontend/**/*`, `deploy/nginx/**/*`, `.gitlab-ci.yml`, `deploy/docker-compose.dev.yml` 변경 시
- 동작 요약:
 1. Node 컨테이너에서 Vite 빌드
 - `VITE_API_ORIGIN`, `VITE_KAKAO_MAP_APP_KEY` 환경 변수 사용
 2. `/tmp/fe_build` → `/srv/frontend/releases/<REL>/` 로 복사
 3. `<REL>/version.json` 생성
 4. `fe_switch.sh` 실행
 - `/srv/frontend/current` 심볼릭 링크를 새 릴리즈로 교체
 - Nginx가 자동으로 신규 버전 사용
 5. `http://nginx/version.json` 호출로 헬스체크
 6. `fe_cleanup.sh` 로 오래된 릴리즈 정리

5-4. Store Assets 배포 (`deploy_dev_store_assets`)

- 트리거 조건: `AssetsStore/**/*` 변경 시.
- 동작 요약

```
mkdir -p /srv/store-assets
```

```
rsync -av --delete AssetsStore/ /srv/store-assets/
```

- Nginx `/store-assets/` 경로로 노출
- 프론트에서 `ENV.ASSETS_ORIGIN` 사용해서 접근